

Agrocarnes Documentaci n del sistema

Inventario, compras, producción, ventas y facturación electrónica

Generado el 2026-09-28 · versión del código: de19491 · 2026-09-28

01 · Descripción general del sistema

Identificación

Nombre del software	Agrocarnes
Tipo	Sistema de información propio (desarrollo a la medida) para inventario, compras, producción, ventas y facturación electrónica
Propietario	Jonatan Botero (desarrollo y administración)
Empresas usuarias	Agrocarnes · Restaurante · D'Monsa Alimentos
Inicio de desarrollo	27 de septiembre de 2026
Puesta en producción	28 de septiembre de 2026
Repositorios	github.com/Ecocargapp/Backend-Agrocarnes · github.com/Ecocargapp/Frontend-Agrocarnes
URL de la aplicación	https://agrocarnes.agrofranpabel.com
URL de la API	https://api-agrocarnes.agrofranpabel.com

Propósito

El sistema controla el flujo físico y de costos de la carne y demás insumos entre tres empresas relacionadas pero jurídicamente independientes, y emite las facturas de venta al cliente final a través de un proveedor tecnológico autorizado (Arco ERP), evitando la doble facturación de un mismo producto.

Empresas y roles en el flujo

Empresa	Rol en el flujo	Relación
Agro Franpabel	Proveedor externo: vende cerdo despostado a Agrocarnes y a D'Monsa con factura	Empresa relacionada (control de granjas en su propio sistema, AgroSoft)
Agrocarnes	Compra la canal, la porciona en cortes, vende al público en mostrador y traslada inventario al Restaurante y a D'Monsa	Sociedad con un socio
Restaurante	Recibe cortes porcionados de Agrocarnes por traslado, los transforma en platos y factura al cliente final	Sociedad con otro socio
D'Monsa Alimentos	Recibe carne para embutidos de Agrocarnes y otros insumos comprados; produce salchicha, jamón, panzerotis, empanadas y palitos de queso	100 % del propietario

Cada empresa es una **entidad independiente dentro del sistema**: tiene sus propias bodegas, sus propios productos, su propio consecutivo de facturación y su propia cuenta ante el proveedor de facturación electrónica. Comparten únicamente la base de datos y el servidor, por economía.

Alcance funcional (versión actual)

Módulo	Función
Inventario	Existencias por bodega con cantidad y costo promedio ponderado; kardex por producto; catálogo de productos
Compras	Registro de la factura del proveedor; entrada de inventario al costo facturado
Traslados	Movimiento de producto entre bodegas de distintas empresas, al costo, con salida y entrada registradas
Formulación y producción	Fórmulas (recetas) por producto; órdenes de producción que consumen insumos y entran el producto terminado con su costo calculado
Ventas	Factura de venta al cliente final con descuento de inventario; envío a facturación electrónica
Facturación electrónica	Emisión a través de Arco ERP (proveedor tecnológico); recepción y almacenamiento del CUFE
Configuración	Parámetros de conexión a Arco por empresa (solo administrador)

Fuera de alcance (a la fecha)

- Contabilidad de partida doble, nómina, cartera y tesorería: se llevan en el ERP contable de cada empresa (Arco) y por el contador.
- Notas crédito y anulación de facturas electrónicas: previstas; se documentarán en la bitácora cuando se implementen.
- Facturación de D'Monsa Alimentos: hoy D'Monsa recibe inventario y produce; sus ventas se facturarán cuando tenga su propia cuenta de Arco.

Relación con el proveedor tecnológico

El sistema **no firma ni transmite directamente** documentos a la DIAN. La factura electrónica la genera, numera, firma y transmite **Arco ERP** (arco365.com), proveedor tecnológico con el que cada empresa tiene su contrato y su resolución de facturación. Agrocarnes actúa como sistema de ventas e inventario que le entrega a Arco la información de cada venta y recibe de vuelta el número de factura y el CUFE. El detalle está en el documento 05.

02 · Arquitectura técnica

Componentes



Componente	Tecnología	Ubicación
Frontend	HTML, CSS y JavaScript (módulos ES) sin framework	/var/www/app-agrocarnes · repo Frontend-Agrocarnes
API	Node.js 20, Express 4, módulos ES	/var/www/agrocarnes-backend · repo Backend-Agrocarnes
Base de datos	PostgreSQL 18, base agrocarnes, usuario agrocarnes_app	Mismo servidor, instalación local
Gestor de procesos	pm2 7 (agrocarnes-api, reinicio automático, arranque con el sistema)	Servidor
Servidor web	nginx con TLS (Let's Encrypt)	Servidor
Servidor	AWS EC2 t3.micro, Ubuntu 26.04, región us-east-2 (Ohio), IP elástica 18.226.158.249	Cuenta AWS del propietario
Facturación electrónica	Arco ERP, API REST v2	Externo (arco365.com)

El mismo servidor aloja el sistema AgroSoft (granjas de Agro Franpabel); los dos sistemas están **completamente separados**: base de datos distinta, proceso pm2 distinto, carpetas distintas, subdominios distintos. Comparten solo la máquina, nginx y PostgreSQL como servicios.

Estructura del código (backend)

src/	
server.js	arranque de Express, rutas, CORS, job de facturación
db/pool.js	conexión a PostgreSQL
db/migrate.js	aplicación de migraciones (tabla schema_migrations)
db/inventario.js	ÚNICA función que mueve inventario (registrarMovimiento)
middleware/auth.js	verificación de JWT y de rol
routes/	un archivo por recurso: auth, empresas, productos, bodegas, terceros, inventario, compras, traslados, recetas, produccion, ventas
dian/arco.js	cliente HTTP de Arco (login, token, reintento en 401)
dian/cliente.js	lógica de facturación electrónica y job de reintentos
migrations/	001_init.sql, 002_..., 003_... (se aplican una sola vez, en orden)
scripts/setup-servidor.sh	instalación/actualización idempotente en el servidor
scripts/crear_admin.js	alta o cambio de clave del administrador
nginx/agrocarnes.conf	configuración base de nginx

Principios de diseño relevantes para auditoría

1. **Un solo punto de entrada al inventario.** Toda variación de existencias pasa por registrarMovimiento, que inserta la fila en movimiento_inventario y actualiza existencia en la **misma transacción**. No hay rutas que modifiquen existencia directamente.

2. **Transacciones.** Compras, traslados, órdenes de producción y ventas se registran en una transacción de base de datos: o queda todo (documento, detalle y movimientos) o no queda nada.
3. **Sin borrado de documentos.** No existen endpoints para eliminar compras, traslados, producciones ni facturas. Las correcciones se hacen con documentos nuevos (y, cuando se implemente, notas crédito).
4. **Trazabilidad.** Cada movimiento de inventario guarda tipo, cantidad, costo, fecha/hora, usuario (`creado_por`) y el documento que lo originó (`referencia_tipo` , `referencia_id`).
5. **Costo promedio ponderado.** Las entradas recalculan el costo promedio; las salidas se valoran al promedio vigente. La fórmula está en `db/inventario.js` y en el documento 04.
6. **Migraciones versionadas.** El esquema de la base de datos solo cambia mediante archivos en `migrations/` , aplicados una vez y registrados en `schema_migrations` con fecha.

Dependencias externas (npm)

Paquete	Uso
express	servidor HTTP / API REST
pg	cliente PostgreSQL
jsonwebtoken	tokens de sesión (JWT)
bcryptjs	hash de contraseñas
cors	control de orígenes permitidos
dotenv	variables de entorno (.env)

Las versiones exactas están fijadas en `package-lock.json` .

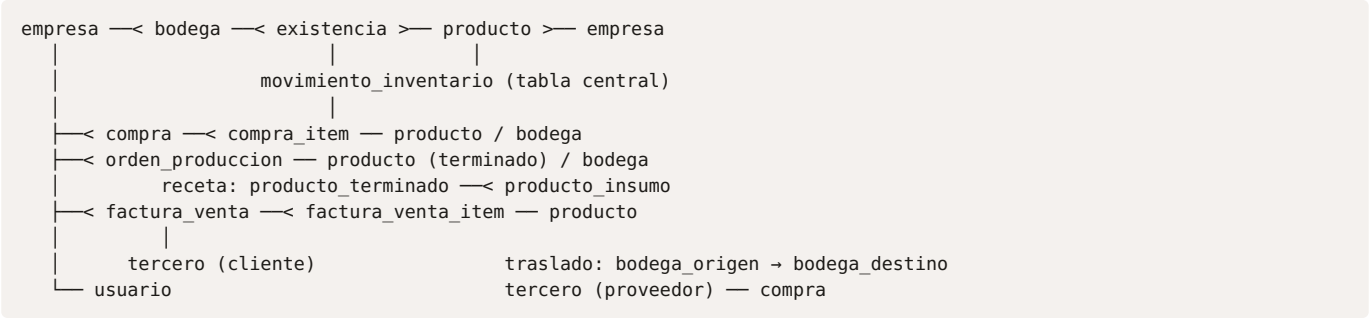
Variables de entorno (`.env` , no versionado)

`PORT` , `PGHOST` , `PGPORT` , `PGDATABASE` , `PGUSER` , `PGPASSWORD` , `JWT_SECRET` , `JWT_EXPIRES_IN` , `CORS_ORIGIN` , `DIAN_JOB` (`off` desactiva el job de facturación, solo para pruebas). Las credenciales de Arco **no** van en `.env` : se guardan por empresa en la base de datos (`empresa.arco_config`).

03 · Modelo de datos

Base de datos PostgreSQL `agrocarnes`. Todas las claves primarias son UUID generados por la base de datos; todos los registros llevan `creado_en` (fecha y hora del servidor, zona UTC). Los importes van en `numeric(14,2)` y las cantidades en `numeric(14,3)` (gramos de precisión en kilogramos).

Diagrama de relaciones



Tablas

empresa

Campo	Tipo	Descripción
id	uuid	PK
nombre	text	Agrocarnes · Restaurante · D'Monsa Alimentos
nit	text	NIT (se diligencia al configurar)
es_facturador_dian	boolean	Emite factura electrónica
prefijo_factura	text	Prefijo del consecutivo interno (AGC, RST, DMS)
ultimo_consecutivo	integer	Último número interno asignado
arco_config	jsonb	Credenciales y parámetros de Arco (ver doc 05)

usuario

Campo	Tipo	Descripción
id, empresa_id	uuid	
nombre, email (único)	text	
password_hash	text	bcrypt, 10 rondas; la contraseña nunca se almacena en claro
rol	text	admin · operador
activo	boolean	Bloqueo de acceso sin borrar el usuario

tercero

Clientes y proveedores. `tipo` ∈ cliente · proveedor · ambos. Campos: `nombre`, `tipo_documento` (CC, NIT, CE...), `numero_documento`, `email`, `telefono`, `direccion`, `ciudad_id` (código DANE), `arco_tercero_id`, `arco_cliente_id` (identificadores del mismo tercero dentro de Arco).

producto

Campo	Tipo	Descripción
empresa_id	uuid	Empresa dueña del producto
nombre	text	Único por empresa
tipo	text	materia_prima · intermedio · terminado
unidad_medida	text	kg, un, lb, g, l
arco_producto_id	text	Código del producto en Arco (obligatorio para facturarlos)
impuesto_pct	numeric(5,2)	% de impuesto incluido en el precio de venta (IVA o imponible)

bodega

empresa_id, nombre. Cada empresa tiene al menos la bodega “Principal”.

existencia

Saldo vivo por (bodega, producto): cantidad, costo_promedio, actualizado_en. Se recalcula en cada movimiento; nunca se edita a mano.

movimiento_inventario — tabla central de auditoría

Campo	Descripción
tipo	compra · porcionado_entrada · porcionado_salida · traslado_salida · traslado_entrada · produccion_consumo · produccion_entrada · venta · devolucion_venta · anulacion_venta
producto_id, bodega_id	Qué y dónde
cantidad	Siempre positiva; el tipo indica si entra o sale
costo_unitario	Costo al que se valoró el movimiento
referencia_tipo, referencia_id	Documento origen: compra · traslado · orden_produccion · factura_venta
creado_por	Usuario que registró el documento
creado_en	Fecha y hora

Con esta tabla se reconstruye el kardex de cualquier producto en cualquier bodega y se verifica que $existencia = \sum \text{entradas} - \sum \text{salidas}$.

compra / compra_item

Cabecera: empresa_id, proveedor_id, numero_factura_proveedor, fecha, total. Detalle: producto_id, bodega_id, cantidad, costo_unitario. Cada ítem genera un movimiento tipo compra.

traslado

producto_id, bodega_origen_id, bodega_destino_id, cantidad, costo_unitario (el promedio de origen en ese momento), es_venta_intercompania, factura_venta_id (si el traslado se factura entre compañías). Genera dos movimientos: traslado_salida y traslado_entrada.

receta

Fórmula: (producto_terminado_id, producto_insumo_id) única → cantidad_por_unidad. Es la cantidad de insumo necesaria para producir **una unidad** del terminado.

orden_produccion

empresa_id, producto_terminado_id, cantidad_producida, bodega_id, costo_unitario (calculado), creado_por. Genera un movimiento `produccion_consumo` por insumo y uno `produccion_entrada` por el terminado.

factura_venta / factura_venta_item

Campo	Descripción
empresa_id, cliente_id	Emisor y adquiriente (nulo = consumidor final)
consecutivo	Número interno al crear; se reemplaza por el número oficial de Arco cuando la factura se emite
fecha, total	
forma_pago	contado · credito
fecha_vencimiento	Solo si es a crédito
saldo	Lo que falta por cobrar; solo lo modifican <code>/cartera/recibos</code> y las notas crédito
estado	vigente · anulada
anulada_en, anulada_por, motivo_anulacion	Trazabilidad de la anulación (directa o por nota crédito razón 2)
estado_dian	pendiente · sin_configurar · enviada · aceptada · error · rechazada
arco_factura_id	Identificador de la factura dentro de Arco
cufe	Código Único de Factura Electrónica devuelto por Arco/DIAN
xml_url, pdf_url	Enlace a la representación publicada por Arco
dian_mensaje, dian_intentos, dian_ultimo_intento, dian_fecha	Trazabilidad del envío

Detalle: producto_id, cantidad, precio_unitario. Cada ítem genera un movimiento `venta` valorado al costo promedio vigente (lo que permite calcular el margen).

compra (cartera)

Igual mecánica que `factura_venta`: `forma_pago` (contado · credito), `fecha_vencimiento`, `saldo` (lo que falta por pagar; solo lo modifica `/cartera/pagos`).

recibo_caja / recibo_caja_aplicacion — cartera de clientes (CxC)

Un recibo de caja es el cobro de una o varias facturas de un mismo cliente. Cabecera (`recibo_caja`): `empresa_id`, `tercero_id` (nulo si es consumidor final), `consecutivo` (por empresa, `empresa.ultimo_recibo`), `fecha`, `medio_pago`, `total`, `notas`. Detalle (`recibo_caja_aplicacion`): `factura_venta_id`, `valor` — puede haber varias aplicaciones por recibo (un pago cubre varias facturas) y varios recibos por factura (pagos parciales). Cada aplicación resta su valor de `factura_venta.saldo`; nunca se permite aplicar más del saldo disponible.

pago_proveedor / pago_proveedor_aplicacion — cartera de proveedores (CxP)

Simétrico al recibo de caja pero para compras: `pago_proveedor` (consecutivo en `empresa.ultimo_pago`) y `pago_proveedor_aplicacion` (`compra_id`, `valor`), que resta de `compra.saldo`.

nota_credito / nota_credito_item

Nota crédito sobre una factura de venta. Cabecera: `empresa_id`, `factura_venta_id`, `consecutivo` (`NC-<prefijo>-<consecutivo>`, en `empresa.ultimo_nota_credito`), `razon` (código DIAN: 1 devolución parcial · 2 anulación · 3 rebaja/descuento · 4 ajuste de precio · 5 dcto pronto pago · 6 dcto volumen), `reingresa_inventario`, `bodega_id` (si reingresa), `total`, `estado de envío a Arco` (`estado_dian`: mismos estados que la factura, más `no_aplica` cuando la factura original no es electrónica), `cufe`, `pdf_url`. Detalle (`nota_credito_item`): `producto_id`, `cantidad`, `precio_unitario`. No se puede devolver más cantidad que la facturada, descontando notas anteriores sobre la misma factura. Si `reingresa_inventario`, genera un movimiento `devolucion_venta` (razones 1,3-6) o `anulacion_venta` (razón 2) por ítem, al costo con el que salió originalmente la venta.

schema_migrations

Registro de qué migraciones se aplicaron y cuándo. Migraciones a la fecha: 001_init.sql (modelo base), 002_bodegas_y_consecutivos.sql (bodega principal por empresa, consecutivos, costo en órdenes de producción), 003_arco.sql (integración con Arco), 004_cartera_y_notas_credito.sql (cuentas por cobrar y por pagar, notas crédito y anulación de facturas).

Reglas de integridad implementadas

- No se permite una salida mayor a la existencia disponible (la transacción se revierte con el mensaje “No hay suficiente existencia para esta salida”).
- Un producto no puede ser insumo de sí mismo.
- Un traslado no puede tener la misma bodega de origen y destino.
- Un usuario inactivo no puede iniciar sesión.
- Las claves foráneas impiden borrar productos, bodegas o terceros que tengan movimientos.

04 · Procesos de negocio

Cada proceso se describe con: quién lo ejecuta, qué pantalla usa, qué queda registrado y cómo afecta el inventario y el costo. Todos los procesos quedan en `movimiento_inventario` con usuario, fecha y documento origen.

4.1 Compra a proveedor

Caso típico: Agro Franpabel vende cerdo despostado a Agrocarnes (o a D'Monsa) y entrega su factura.

1. Pantalla **Compras**. El operador elige la empresa que compra, la bodega que recibe, el proveedor (o lo crea) y digita el número de la factura del proveedor, la fecha y las líneas (producto, cantidad, costo unitario).
2. El sistema guarda `compra` + `compra_item` y, por cada línea, un movimiento `compra` en la bodega indicada.
3. **Efecto en inventario:** entra la cantidad; el costo promedio se recalcula:

$$\text{costo_promedio_nuevo} = (\text{stock_anterior} \times \text{costo_anterior} + \text{cantidad} \times \text{costo_compra}) \div (\text{stock_anterior} + \text{cantidad})$$

4. **Soporte:** la factura física/electrónica del proveedor, cuyo número queda en `numero_factura_proveedor`.

4.2 Porcionado (despiece) y producción por fórmula

El porcionado de la canal y la producción de embutidos y productos de panadería se registran con el mismo mecanismo: una **fórmula** que dice cuánto insumo se necesita por unidad de producto terminado.

Ejemplos de fórmulas: - Lomo de cerdo (kg) = 1,25 kg de cerdo despostado (rendimiento 80 %). - Carne para embutido (kg) = 1,00 kg de cerdo despostado. - Salchicha (kg) = 0,90 kg de carne para embutido + condimentos.

1. Pantalla **Formulación → Fórmulas**: se define o ajusta la fórmula del producto. Queda en `receta`.
2. Pantalla **Formulación → Producir**: el operador indica producto, bodega y cantidad producida. El sistema muestra antes de confirmar cuánto insumo va a consumir.
3. Se guarda `orden_produccion`; por cada insumo un movimiento `produccion_consumo` (cantidad = cantidad_por_unidad × cantidad producida, valorado al costo promedio del insumo) y un movimiento `produccion_entrada` del terminado.
4. **Costo del terminado** = $\Sigma (\text{insumo consumido} \times \text{su costo promedio}) \div \text{cantidad producida}$. Queda en `orden_produccion.costo_unitario` y entra al promedio del producto terminado.

La merma queda implícita en la fórmula (rendimiento). Si el rendimiento real difiere, se ajusta la fórmula; el historial de órdenes conserva el costo con el que se produjo cada lote.

4.3 Traslado entre empresas

Casos: Agrocarnes → Restaurante (cortes porcionados) y Agrocarnes → D'Monsa (carne para embutidos).

1. Pantalla **Traslados**: bodega de origen, producto (solo aparecen los que tienen existencia allí), bodega de destino y cantidad.
2. Se guarda `traslado` con el costo promedio de origen en ese momento, y dos movimientos: `traslado_salida` en origen y `traslado_entrada` en destino, ambos al mismo costo.
3. **Efecto:** la existencia baja en una empresa y sube en la otra **al costo**, sin precio de venta y sin factura. Así el producto se factura una sola vez: cuando la empresa destino lo vende al cliente final.
4. El campo `es_venta_intercompania` permite marcar el traslado como venta entre compañías y asociarle una factura, si el tratamiento tributario definido con el contador así lo requiere. **A la fecha los traslados se registran como movimiento interno sin factura; este punto está pendiente de confirmación con el contador** (ver CHANGELOG).

4.4 Venta al cliente final

Casos: mostrador de Agrocarnes; consumo en el Restaurante.

1. Pantalla **Ventas**: punto de venta (empresa/bodega), cliente (opcional; sin cliente = consumidor final) y líneas (producto con existencia, cantidad, precio unitario).
2. En una transacción se guarda `factura_venta` (con consecutivo interno provisional), `factura_venta_item` y un movimiento `venta` por línea, valorado al costo promedio (para calcular margen). Si algún producto no tiene existencia suficiente, la venta completa se rechaza.
3. Inmediatamente después, y **sin bloquear la venta**, el sistema envía la factura al proveedor de facturación electrónica (doc 05). Cuando Arco la emite, el consecutivo provisional se reemplaza por el **número oficial de la factura electrónica** (prefijo y número de la resolución de la empresa) y se guarda el CUFE.
4. El estado de cada factura frente a la DIAN es visible en la lista de ventas; el detalle muestra CUFE, mensaje y permite reintentar.

4.4bis Cartera (cuentas por cobrar y por pagar)

Toda venta y toda compra nace con `forma_pago` contado o crédito. De contado se cobra/paga de inmediato (recibo o pago automático por el mismo valor); a crédito queda con `saldo` pendiente y una `fecha_vencimiento` (plazo en días o fecha explícita).

1. Pantalla **Cartera → Por cobrar**: un resumen por cliente con antigüedad de saldos (por vencer, 1-30, 31-60, 61-90, más de 90 días) y, al entrar a un cliente, el detalle de sus facturas con saldo. El botón **Recibo de caja** registra un cobro (efectivo, transferencia, tarjeta u otro) que se puede aplicar a una o varias facturas del mismo cliente en un solo movimiento; no se permite aplicar más del saldo de cada factura.
2. Pantalla **Cartera → Por pagar**: lo mismo para proveedores. El botón **Pago a proveedor** aplica a una o varias compras.
3. El saldo de cada factura/compra vive únicamente en `factura_venta.saldo` / `compra.saldo` y solo lo modifican estos dos flujos (y las notas crédito, 4.4ter). Nunca se edita a mano.
4. Pantalla **Cartera → Resumen**: total por cobrar y por pagar, y cuánto de eso está vencido, por empresa.

4.4ter Notas crédito y anulación de facturas

Corrige una factura de venta ya registrada sin borrar ni editar el documento original (ver 4.6). Pantalla **Ventas → detalle de factura → Nota crédito**, con una razón (código DIAN):

Razón	Efecto en inventario	Efecto en cartera
1 · Devolución parcial	Reingresa lo devuelto (opcional)	Reduce el saldo por cobrar en el valor de la nota
2 · Anulación de factura	Reingresa todo lo vendido	Salda la factura y la deja <code>anulada</code>
3 · Rebaja o descuento	No mueve inventario	Reduce el saldo por cobrar
4 · Ajuste de precio	No mueve inventario	Reduce el saldo por cobrar
5 · Descuento pronto pago	No mueve inventario	Reduce el saldo por cobrar
6 · Descuento por volumen	No mueve inventario	Reduce el saldo por cobrar

No se puede devolver o descontar más de lo facturado (se valida contra las notas ya emitidas sobre la misma factura). Si la factura ya tenía cobros aplicados y se anula, el sistema informa el saldo a favor del cliente en lugar de aplicarlo automáticamente. Cuando la factura fue emitida electrónicamente, la nota crédito se envía a Arco en segundo plano (doc 05); si no lo fue, el efecto es solo local.

Anular una factura **antes** de emitirla electrónicamente (`arco_factura_id` nulo) usa en cambio `POST /ventas/:id/anular`, que revierte directamente sin generar una nota crédito formal — solo aplica si no tiene cobros aplicados a crédito.

4.5 Consulta de inventario y kardex

Pantalla **Inventario**: existencias por bodega (cantidad, costo promedio, valor) y, al seleccionar un producto, su kardex: cada movimiento con fecha, tipo, cantidad (+/–) y costo. El valor del inventario mostrado es $\Sigma \text{cantidad} \times$

costo_promedio.

4.6 Correcciones

No se editan ni eliminan documentos ya registrados. Una compra mal digitada se corrige con una compra de ajuste; un traslado en sentido contrario devuelve el producto; una venta se corrige con una nota crédito (4.4ter) o, si aún no ha sido emitida electrónicamente y no tiene cobros a crédito aplicados, con `POST /ventas/:id/anular`. Cualquier ajuste manual que fuera indispensable fuera de estos flujos se hace en la base de datos por el administrador y se documenta en el CHANGELOG con fecha, motivo y registros afectados.

05 · Facturación electrónica

Modelo de operación

Rol	Quién	Qué hace
Sistema de ventas e inventario	Agrocarnes (este software)	Registra la venta, descuenta inventario, entrega a Arco los datos de la factura, recibe y guarda número oficial y CUFE
Proveedor tecnológico / software de facturación	Arco ERP (arco365.com)	Numera con la resolución DIAN de la empresa, genera el XML UBL, firma digitalmente, transmite a la DIAN, publica la representación gráfica y devuelve el CUFE
Obligado a facturar	Cada empresa (Agrocarnes, Restaurante) con su NIT, su resolución y su cuenta de Arco	Responsable ante la DIAN

Este software **no genera XML, no firma y no se comunica con los servicios de la DIAN**. Toda factura electrónica existe primero en Arco, que es quien la reporta. Agrocarnes conserva copia de los datos comerciales (cliente, ítems, valores), el identificador de Arco, el número oficial y el CUFE, lo que permite cruzar en cualquier momento las ventas del sistema contra las facturas emitidas en Arco y reportadas a la DIAN.

Flujo detallado

```
Venta registrada (estado_dian = pendiente)
|
▼ enviarFacturaADian() — inmediato tras la venta y reintentos cada 2 min
├ 1. Login en Arco (token OAuth, se renueva automáticamente)
├ 2. Cliente: ClienteId mapeado · o consumidor final (cliente_default_id)
│   · o alta automática en Arco (Tercero/Insert + Cliente/Insert)
├ 3. Ítems: cada producto debe tener arco_producto_id;
│   precio → valor base (si el precio incluye impuesto, se divide por 1 + %)
├ 4. POST Factura/Insert → { FacturaId } estado = enviada
▼
sincronizarEstado() — GET Factura/Get/{FacturaId}
├ FacturaId2 (prefijo+número oficial) → consecutivo
├ FacturaCUFE presente → estado = aceptada, se guarda cufe, url, fecha DIAN
└ FacturaMsgFE con error → estado = rechazada, se guarda el mensaje
```

Errores técnicos (red, credenciales, Arco caído) dejan la factura en estado `error` con el mensaje en `dian_mensaje`; el job la reintenta hasta 12 veces. Un producto sin código Arco también queda en `error` y se reintenta solo en cuanto se asigna el código. Nada de esto impide que la venta y el descuento de inventario ya estén registrados.

Estados de una factura (estado_dian)

Estado	Significado	Acción
pendiente	Registrada, aún no enviada	Automática
sin_configurar	La empresa no tiene cuenta de Arco configurada	Configurar y reintentar
enviada	Creada en Arco, esperando CUFE	Automática (consulta cada 5 min)
aceptada	CUFE recibido	Ninguna
error	Falló el envío por causa técnica o de datos	Se reintenta sola; el mensaje indica la causa
rechazada	Arco/DIAN la rechazó o se agotaron los reintentos	Revisar el mensaje; corregir y reintentar manualmente

Datos enviados a Arco por factura

Campo Arco	Origen en Agrocarnes
DocumentId, FacturaResolucionTipo	Configuración de la empresa (tipo de documento / resolución en Arco)
FacturaFecha	Fecha del día de emisión (zona America/Bogota)
ClientId	Cliente mapeado o consumidor final
FacturaBodegald, FacturaSucursalId, VendedorId, FacturaTipoPago	Configuración de la empresa
FacturaOrdenCliente	Consecutivo interno de Agrocarnes (para cruce)
FacturaOrdenInterna	UUID de la factura en Agrocarnes (para cruce)
Detalle[].ProductId	<code>producto.arco_producto_id</code>
Detalle[].FacturaDetalleCantidad	Cantidad vendida
Detalle[].FacturaDetalleVrUnitario	Precio unitario base (sin impuesto)

Los impuestos (IVA, imptoconsumo) los liquida Arco según la configuración tributaria de cada producto en Arco. En Agrocarnes, `impuesto_pct` solo se usa para convertir el precio de venta (que incluye impuesto) a valor base.

Datos recibidos de Arco y conservados

`arco_factura_id` (id interno de Arco), `consecutivo` (número oficial, ej. AGC1234), `cupe`, `xml_url/pdf_url` (representación gráfica publicada por Arco), `dian_fecha`, `dian_mensaje`.

Configuración por empresa (`empresa.arco_config`)

Se administra en la pantalla **Configuración** (solo rol admin) e incluye: host de Arco, nombre de empresa en Arco, usuario, contraseña (se guarda en la base de datos, nunca se devuelve al navegador), DocumentId, tipo de resolución, SucursalId, Bodegald, ClientId de consumidor final, VendedorId, tipo de pago, ciudad DANE por defecto para clientes nuevos y si los precios incluyen impuesto. El botón *Probar conexión* valida las credenciales y muestra los catálogos de Arco (sucursales, bodegas, documentos en uso) para diligenciar los parámetros sin error.

Referencia de la API de Arco utilizada

Documentación oficial: <https://documenter.getpostman.com/view/289978/UzJFweL6>

Endpoint	Uso
POST Security/Login	Obtener token
POST Factura/Insert	Crear la factura de venta
GET Factura/Get/{id}	Consultar número oficial, CUFE, estado FE
GET Factura/List	Prueba de conexión (documentos en uso)
GET Cliente/List, POST Tercero/Insert, POST Cliente/Insert	Buscar o crear el adquiriente
GET Sucursal/List, GET Bodega/List	Prueba de conexión

POST NotaCredito/Insert | Crear la nota crédito (devolución, rebaja, ajuste o anulación) |
GET NotaCredito/Get/{id} | Consultar CUFE y estado FE de la nota crédito |

Nota sobre NotaCredito/Insert y NotaCredito/Get: a diferencia de Factura/Insert (ya probado contra el simulador de Arco), los nombres exactos de los campos de estos dos endpoints no están confirmados contra la documentación pública de Arco (no la expone con ese nivel de detalle); `src/dian/notas.js` los implementó siguiendo la misma convención de nombres que usa Factura/Insert del mismo proveedor. **Antes de la primera nota crédito real hay que confirmarlos con soporte de Arco (o su Postman privado) y probarlos contra el simulador,**

igual que se hizo con la factura.

Notas crédito y anulación de facturas

Cubre devoluciones, rebajas/descuentos, ajustes de precio y anulación de una factura ya emitida electrónicamente (POST /notas-credito, ver doc 04 y 03).

- Si la factura original **no** fue emitida electrónicamente (arco_factura_id nulo, por ejemplo porque la empresa no factura electrónicamente o el envío nunca se hizo), la nota crédito queda con estado_dian = 'no_aplica' y nunca se envía a Arco: el efecto es puramente local (inventario y cartera).
- Si sí fue emitida, la nota se crea local en la misma transacción y, ya confirmada, se envía a Arco en segundo plano (enviarNotaCreditoADian), con el mismo patrón de reintentos y estados que la factura (pendiente → enviada → aceptada/rechazada, job cada 2 min).
- Razón 2 (anulación) siempre reingresa el inventario completo de la factura y la deja en estado = 'anulada'; las demás razones son parciales y no cambian el estado de la factura, solo su saldo por cobrar.

Cruce para auditoría

Para verificar que toda venta del sistema tiene su factura electrónica:

```
select consecutivo, fecha, total, estado_dian, cufe, arco_factura_id
from factura_venta
where empresa_id = '<empresa>'
and fecha between '<desde>' and '<hasta>'
order by fecha;
```

Toda fila debería estar en aceptada con cufe no nulo. Las que no lo estén tienen en dian_mensaje la causa y son las que requieren gestión.

Factus (nuevo proveedor, en transición desde Arco)

Se agregó Factus (developers.factus.com.co) como segundo proveedor de facturación y nómina electrónica, para reemplazar a Arco. Se implementó **en paralelo**, sin quitar Arco: cada empresa elige su proveedor activo con empresa.proveedor_dian (arco | factus, pantalla Configuración), así se puede probar o migrar una empresa a la vez sin afectar a las demás.

- src/dian/factus.js: cliente HTTP con autenticación OAuth2 (grant password, refresh con refresh_token), igual patrón que arco.js.
- src/dian/facturas-factus.js / src/dian/notas-factus.js: arman el JSON de Factus y llaman POST /v2/bills/validate / POST /v2/credit-notes/validate. A diferencia de Arco, Factus **valida y firma en la misma llamada** — normalmente ya devuelve el CUFE de una vez, sin necesitar un paso de sincronización aparte (aunque sincronizarEstado/sincronizarNotaCredito existen por si Factus responde sin CUFE y hay que consultarlo después).
- src/dian/cliente.js y src/dian/notas.js ahora son **despachadores**: enviarFacturaADian/sincronizarEstado (y sus equivalentes de notas) consultan empresa.proveedor_dian y delegan a la implementación de Arco o de Factus; el job en segundo plano (iniciarJobDian, iniciarJobNotasCredito) recorre empresas con cualquiera de los dos proveedores configurados.
- El reference_code que se envía a Factus es simplemente el id local de la factura/nota (AGC-<id> / AGC-NC-<id>), lo que hace el envío idempotente ante reintentos sin necesitar columnas nuevas; el “número” que Factus asigna (ej. SETP990021506, CRTE713) se guarda en la columna genérica consecutivo que ya existía para Arco.
- Nuevas columnas: empresa.factus_config (credenciales, rangos de numeración, cliente por defecto), empresa.proveedor_dian, producto.factus_unidad_medida_code (código UN/CEFACT; si está vacío se infiere de unidad_medida) y producto.factus_estandar_code (código de bien/servicio, 999 = no aplica por defecto).

Probado contra el sandbox real de Factus (api-sandbox.factus.com.co, 28/09/2026): login OAuth2, GET /v2/numbering-ranges (rango 389 = facturas, 1776 = notas crédito, cuenta de pruebas compartida), POST /v2/bills/validate con CUFE real devuelto de inmediato, y POST /v2/credit-notes/validate anulando esa factura (número CRTE713), correctamente enlazada a través de bill_number. También se verificó con una prueba local (sin red, con las respuestas reales capturadas) que src/dian/cliente.js y notas.js arman el payload correcto (ítems, cliente,

forma de pago, rangos de numeración) y escriben el estado esperado en la base de datos.

Pendiente antes de facturar de verdad con Factus: - Reemplazar las credenciales de prueba (`sandboxv2@factus.com.co`) por las credenciales reales de cada empresa cuando Factus las entregue, y cambiar `base_url` a `https://api.factus.com.co` en Configuración. - Confirmar `payment_method_code` para ventas a crédito (se usa "1" por ahora, sin verificar contra la tabla completa de medios de pago de Factus) y completar la tabla de conversión Colombia-DIAN si aparecen más casos (retenciones, IVA excluido vs. 0%, etc.). - Revisar `factus_unidad_medida_code` de los productos que no sean `kg/un` / `lb/g/l` (el valor por defecto 94 = unidad es un comodín razonable pero no siempre exacto). - Nómina electrónica de Factus: no se implementó en esta iteración (solo facturación de venta y notas crédito); queda para cuando se decida reemplazar también la nómina.

06 · Seguridad y control de acceso

Autenticación

- Inicio de sesión con correo y contraseña contra la tabla `usuario`.
- Las contraseñas se almacenan únicamente como hash **bcrypt** (10 rondas); no existe forma de recuperar una contraseña, solo de reemplazarla (`scripts/crear_admin.js`).
- Tras el inicio de sesión el servidor emite un **JWT** firmado con `JWT_SECRET` (secreto aleatorio de 96 caracteres hexadecimales generado en la instalación), con vigencia de 12 horas. El token viaja en el encabezado `Authorization: Bearer ...` y se guarda en el navegador del usuario.
- Un token inválido o vencido devuelve 401 y la aplicación cierra la sesión.
- Los usuarios inactivos (`activo = false`) no pueden autenticarse.

Autorización

Rol	Puede
operador	Registrar compras, traslados, producciones y ventas; consultar inventario; crear productos, clientes y proveedores
admin	Todo lo anterior + configurar la cuenta de Arco de cada empresa (<code>/empresas/:id/arco</code>)

Todas las rutas de la API, salvo `/auth/login` y `/health`, exigen un token válido (`requireAuth`); las de configuración exigen además rol admin (`requireRole('admin')`).

Transporte y red

- Toda la comunicación navegador ↔ servidor va por **HTTPS** con certificados Let's Encrypt renovados automáticamente por Certbot; HTTP redirige a HTTPS.
- La API solo acepta peticiones del origen de la aplicación (CORS restringido a `https://agrocarnes.agrofrapabel.com`).
- PostgreSQL escucha únicamente en `localhost`; no es accesible desde Internet. El usuario `agrocarnes_app` solo tiene privilegios sobre la base `agrocarnes`.
- El acceso administrativo al servidor es por SSH con llave (sin contraseña) o por la consola de AWS con autenticación de la cuenta.

Secretos

Secreto	Dónde vive	Quién lo ve
Contraseña de la base de datos	<code>/var/www/agrocarnes-backend/.env</code> (permisos 600)	Administrador del servidor
JWT_SECRET	<code>.env</code>	Administrador del servidor
Credenciales de Arco	<code>empresa.arco_config</code> en la base de datos	Se escriben desde la pantalla de Configuración; la API nunca devuelve la contraseña
Contraseñas de usuarios	Solo hash bcrypt	Nadie

El archivo `.env` está excluido del control de versiones (`.gitignore`). El repositorio no contiene ningún secreto.

Integridad y trazabilidad de la información

- **Sin borrado:** la API no expone eliminación de compras, traslados, órdenes de producción, facturas ni movimientos de inventario.
- **Autoría:** cada movimiento de inventario registra el usuario (`creado_por`) y la fecha/hora del servidor.

- **Consecutivos:** el número interno de factura se asigna dentro de una transacción con bloqueo de fila (`update ... returning`), por lo que no puede repetirse ni saltarse por concurrencia. El número oficial lo asigna Arco con la resolución DIAN.
- **Historial del código:** todo cambio al software queda en Git con fecha, autor y descripción; los repositorios son públicos y su historial es verificable por terceros.
- **Migraciones:** los cambios de estructura de datos quedan registrados en `schema_migrations` con fecha de aplicación.

Registro de accesos y actividad

A la fecha el sistema registra la autoría de cada documento (usuario) pero **no** lleva una bitácora separada de inicios de sesión ni de consultas. Está previsto como mejora (ver CHANGELOG · pendientes).

Gestión de usuarios

Los usuarios se crean con `scripts/crear_admin.js` (rol admin) o directamente en la tabla `usuario` por el administrador. La pantalla de administración de usuarios está prevista como mejora. Cuando un empleado se retira, se marca `activo = false`; el historial de sus movimientos se conserva.

07 · Operación, respaldos y continuidad

Despliegue y actualización

La instalación y cada actualización se hacen con **un solo comando**, desde la terminal del servidor (usuario `ubuntu`):

```
bash <(curl -fsSL https://raw.githubusercontent.com/Ecocargapp/Backend-Agrocarne/main/scripts/setup-servidor.sh) correo-del-admin@dominio.com
```

El script (`scripts/setup-servidor.sh`) es **idempotente**: se puede repetir sin efectos secundarios. En cada ejecución:

1. Verifica herramientas (node, npm, pm2, psql, nginx).
2. Crea la base de datos y el usuario si no existen (con contraseña aleatoria).
3. Sincroniza el código del backend con la rama `main` de GitHub.
4. Genera `.env` la primera vez (secretos aleatorios); después lo conserva.
5. Instala dependencias y aplica las migraciones pendientes.
6. Crea el administrador (o fija su clave si se pasa como 2.º argumento).
7. Arranca o reinicia `agrocarnes-api` en pm2 y comprueba `/health`.
8. Instala el respaldo diario en cron y ejecuta uno de inmediato.
9. Sincroniza el frontend.
10. Instala la configuración de nginx la primera vez (después la conserva, porque Certbot la modifica) y recarga nginx.
11. Emite o reinstala el certificado HTTPS cuando el DNS apunta al servidor.

Toda versión desplegada corresponde a un commit identificable en GitHub (`git -C /var/www/agrocarnes-backend log -1`).

Respaldos

Qué	Cómo	Dónde	Retención
Base de datos	<code>pg_dump</code> comprimido, diario 02:30 (cron), <code>scripts/respaldo-bd.sh</code>	<code>/var/backups/agrocarnes/agrocarnes-AAAAMDD-HHMMSS.sql.gz</code> (permisos 600)	30 días
Código y documentación	Git, en GitHub	github.com/Ecocargapp	Permanente (historial completo)
Configuración (<code>.env</code> , nginx, certificados)	Manual	Servidor	—

Registro de ejecuciones: `/var/log/agrocarnes-respaldo.log`.

Pendiente (recomendado): copia diaria de los respaldos fuera del servidor (por ejemplo a un bucket S3 con versionado) para cubrir la pérdida total de la instancia. Ver CHANGELOG · pendientes.

Restauración

```
# en el servidor, con la API detenida
pm2 stop agrocarnes-api
sudo -u postgres psql -c "drop database agrocarnes;" -c "create database agrocarnes owner agrocarnes_app;"
gunzip -c /var/backups/agrocarnes/agrocarnes-AAAAMDD-HHMMSS.sql.gz | sudo -u postgres psql -d agrocarnes
pm2 start agrocarnes-api
```

Para reconstruir el servidor completo desde cero: crear una instancia Ubuntu con node, pm2, PostgreSQL, nginx y certbot; correr el script de despliegue; restaurar el último respaldo; apuntar el DNS.

Monitoreo y registros

Qué	Comando
Estado de los procesos	<code>pm2 ls</code>
Registro de la API (errores, envíos a Arco)	<code>pm2 logs agrocarnes-api</code>
Salud de la API	<code>curl https://api-agrocarnes.agrofranpabel.com/health</code>
nginx	<code>sudo nginx -t, /var/log/nginx/access.log, error.log</code>
Certificado	<code>sudo certbot certificates</code>
Facturas con problema DIAN	<code>select consecutivo, estado_dian, dian_mensaje from factura_venta where estado_dian in ('error', 'rechazada', 'sin_configurar');</code>

pm2 reinicia la API si falla y la levanta al reiniciar el servidor (`pm2 save`).

Entorno de pruebas

El repositorio permite levantar todo en un computador local (README): PostgreSQL local, `npm run migrate`, `npm run dev`, frontend servido con cualquier servidor estático. Con `DIAN_JOB=off` no se envía nada a Arco. Las pruebas previas a cada entrega se ejecutan así, contra un simulador de la API de Arco, antes de desplegar.

Responsables

Actividad	Responsable
Desarrollo, despliegue, administración del servidor	Jonatan Botero
Operación diaria (compras, producción, ventas)	Personal de cada empresa con usuario propio
Facturación electrónica (resolución, certificado, contrato)	Cada empresa con Arco
Revisión contable y tributaria	Contador de cada empresa

Bitácora de cambios — Agrocarnes

Registro cronológico de todas las versiones del software. Cada entrada corresponde a uno o más commits en GitHub (Backend-Agrocarnes y Frontend-Agrocarnes), verificables con `git log`. Formato: fecha · versión · qué cambió · por qué · migraciones de base de datos.

Convención de versiones: AAAA.MM.DD.n (fecha de la entrega y número de entrega del día).

2026.09.28.8 — Factus como nuevo proveedor de facturación electrónica

- Backend: se agrega **Factus** (`developers.factus.com.co`) en paralelo a Arco, sin quitarlo: cada empresa elige su proveedor activo (`empresa.proveedor_dian`, `arco` | `factus`). `src/dian/factus.js` (cliente OAuth2), `src/dian/facturas-factus.js` y `src/dian/notas-factus.js` (envío de facturas y notas crédito). `src/dian/cliente.js` y `notas.js` ahora despachan a Arco o a Factus según la empresa; el job de reintentos cubre ambos proveedores.
- Backend: GET/PUT/DELETE `/empresas/:id/factus` (configuración) y POST `/empresas/:id/factus/probar` (prueba credenciales y lista los rangos de numeración disponibles); PUT `/empresas/:id/proveedor-dian` para elegir el proveedor activo.
- Frontend: pantalla **Configuración** ahora tiene un selector de proveedor por empresa y el formulario de cuenta de Factus (credenciales, entorno sandbox/producción, rangos de numeración, botón “Probar conexión”); en **Inventario**, columna editable para el código de unidad de medida de Factus por producto (opcional; si está vacío se infiere de la unidad).
- Migración `005_factus.sql`: `empresa.factus_config`, `empresa.proveedor_dian`, `producto.factus_unidad_medida_code`, `producto.factus_estandar_code`.
- Probado contra el **sandbox real de Factus** con las credenciales de prueba que dio Factus: login OAuth2, GET `/v2/numbering-ranges`, creación de una factura con CUFE real devuelto de inmediato, y una nota crédito de anulación referenciándola correctamente. También se verificó localmente (sin red, con las respuestas reales capturadas) que el payload armado por `cliente.js/notas.js` es correcto y que el estado queda bien guardado en la base de datos. Detalle y pendientes antes de facturar de verdad con Factus en docs/05.

2026.09.28.7 — Pantallas de cartera y notas crédito; informes a Excel

- Frontend: pantalla **Cartera** (por cobrar / por pagar) con resumen, antigüedad de saldos por cliente/proveedor, detalle de documentos y registro de recibos de caja y pagos a proveedores (aplicables a una o varias facturas/compras a la vez).
- Frontend: pantalla **Notas crédito**, accesible también desde el detalle de una factura en Ventas (“Nota crédito / Anular”): selecciona la factura, marca los productos y cantidades a devolver con su razón, o anula la factura completa con un solo botón.
- Frontend: Ventas y Compras ahora piden la forma de pago (contado/crédito) al registrar el documento, con medio de pago o plazo según corresponda; antes siempre se enviaban como “contado” sin que la pantalla lo pidiera.
- Frontend: **informe de venta diaria** (Ventas) y **detalle de cartera** (Cartera) descargables a Excel (.xlsx), con fecha de factura, fecha de vencimiento, forma y medio de pago. Se generan en el navegador con la librería `xlsx` (SheetJS) empaquetada localmente en `js/vendor/` — no se carga desde un CDN externo, para no depender de la red de cada cliente.
- Backend: GET `/ventas/reporte-diario` (ventas del día con forma y medio de pago) y GET `/cartera/clientes/documentos` / GET `/cartera/proveedores/documentos` (todos los documentos con saldo, con fecha de factura y vencimiento) para alimentar esos informes. GET `/ventas/:id` ahora también devuelve `producto_id` en cada ítem (lo necesita la pantalla de notas crédito).
- Corrección: un `[hidden]` en un `<label>` no ocultaba nada porque la regla propia `label { display: flex }` le ganaba a `[hidden]` del navegador (el origen “author” siempre gana sobre el “user-agent”, sin importar especificidad) — afectaba a los campos que se muestran u ocultan según la forma de pago elegida. Se agregó `[hidden] { display: none !important; }` una sola vez en `styles.css`.
- Pruebas: flujo de punta a punta con un navegador real (Playwright, sin interfaz) contra la API local — venta de contado y a crédito, alternar forma de pago, descarga y verificación del contenido de los dos informes Excel, recibo y pago parciales con la validación de saldo, anulación de factura por nota crédito, y las notas crédito

quedando en `no_aplica` cuando la factura no es electrónica.

2026.09.28.6 — Cartera propia y notas crédito

- Backend: cartera de cuentas por cobrar y por pagar propia del sistema (no depende de Arco ni del reporte del contador): recibos de caja (cobro de una o varias facturas de un cliente) y pagos a proveedores (una o varias compras), con antigüedad de saldos (por vencer, 1-30, 31-60, 61-90, más de 90 días) y resumen por empresa (`src/routes/cartera.js`).
- Backend: notas crédito sobre facturas de venta — devolución parcial, anulación (reingresa todo el inventario y anula la factura), rebaja o descuento, ajuste de precio, descuento pronto pago y por volumen; valida que no se devuelva más de lo facturado descontando notas anteriores (`src/routes/notas-credito.js`). Envío a Arco en segundo plano cuando la factura original fue emitida electrónicamente (`src/dian/notas.js`, mismo patrón de reintentos que las facturas); si no lo fue, el efecto es solo local (`estado_dian = 'no_aplica'`).
- Ventas y compras ahora admiten `forma_pago` contado/crédito con plazo o fecha de vencimiento; de contado se cobra/paga automáticamente al registrar el documento.
- Corrección: el envío de `anulacion: true` sin razón explícita en `POST /notas-credito` fallaba la validación antes de inferir `razon = 2` (se movió la inferencia antes de validar).
- Migración `004_cartera_y_notas_credito.sql`: `factura_venta.forma_pago`, `fecha_vencimiento`, `saldo`, `estado`, `anulada_en`, `anulada_por`, `motivo_anulacion`, `creado_por`; `compra.forma_pago`, `fecha_vencimiento`, `saldo`, `creado_por`; `tablas recibo_caja`, `recibo_caja_aplicacion`, `pago_proveedor`, `pago_proveedor_aplicacion`, `nota_credito`, `nota_credito_item`; `empresa.ultimo_recibo`, `ultimo_pago`, `ultimo_nota_credito`; nuevos tipos de movimiento de inventario `devolucion_venta` y `anulacion_venta`.
- Pruebas: flujo completo probado de punta a punta contra una base de datos local (compra a crédito → pago parcial → saldo correcto; venta a crédito → recibo parcial → saldo correcto; nota crédito con devolución parcial → reingreso de inventario al costo original y reducción del saldo; nota crédito de anulación total → saldo en cero, factura anulada, inventario reingresado; rechazo de recibos/pagos que superan el saldo disponible). Pendiente: validar los campos de `NotaCredito/Insert` contra Arco antes de la primera nota crédito real (ver README y docs/05).
- docs/03, docs/04 y docs/05 actualizados con el modelo de datos, el proceso de cartera y notas crédito, y el detalle de la integración con Arco.

2026.09.28.5 — Documentación para auditoría y respaldos

- Se crea la carpeta `docs/` con la documentación técnica y funcional completa (descripción general, arquitectura, modelo de datos, procesos, facturación electrónica, seguridad, operación) y esta bitácora.
- Se agrega `scripts/generar-pdf.sh` para producir el PDF de la documentación.
- Se agrega el **respaldo diario automático** de la base de datos (`scripts/respaldo-bd.sh`, cron 02:30, 30 días de retención) y su instalación en `setup-servidor.sh`.
- Regla adoptada: todo cambio funcional actualiza `docs/` y esta bitácora en el mismo commit.
- Sin migraciones.

2026.09.28.4 — Corrección: conservar HTTPS en despliegues

- `setup-servidor.sh` ya no sobrescribe la configuración de nginx cuando Certbot la ha modificado, y reinstala el certificado si existe pero nginx no lo usa. Motivo: un despliegue había dejado el sitio sin HTTPS.
- Sin migraciones.

2026.09.28.3 — Facturación electrónica vía Arco ERP

- Backend: cliente de la API de Arco (`src/dian/arco.js`), envío de facturas (`Factura/Insert`), consulta de número oficial y CUFE (`Factura/Get`), búsqueda/alta automática de clientes en Arco, job de reintentos cada 2 minutos, endpoints de configuración y prueba de conexión por empresa, reenvío manual y actualización de estado por factura.
- Frontend: pantalla **Configuración** (solo admin), código Arco y % de impuesto en productos (creación y edición en línea), estados DIAN, CUFE, mensaje y botones de reintento en Ventas; dirección y ciudad en clientes.
- Motivo: emitir factura electrónica a la DIAN a través del proveedor tecnológico Arco, con el que cada empresa

tiene su resolución.

- Migración `003_arco.sql`: `empresa.arco_config`; `producto.arco_producto_id`, `producto.impuesto_pct`; `tercero.arco_tercero_id`, `arco_cliente_id`, `ciudad_id`, `direccion`; `factura_venta.arco_factura_id`, `dian_mensaje`, `dian_intentos`, `dian_ultimo_intento`, `dian_fecha`.
- Estado: probado contra un simulador de la API de Arco; pendiente la primera factura real cuando se configuren las credenciales de Arco de Agrocarnes y del Restaurante.

2026.09.28.2 — Módulos operativos completos

- Backend: rutas de bodegas, terceros (clientes/proveedores) y recetas; listados con detalle de compras, traslados, producción y ventas; consecutivo interno de factura por empresa (AGC-, RST-, DMS-); costo unitario guardado en cada orden de producción; CORS tolerante.
- Frontend: módulos **Inventario** (existencias por bodega, kardex, alta de productos), **Compras** (líneas múltiples, alta de proveedor), **Traslados** (según existencia de origen), **Formulación** (fórmulas y producción con vista previa de consumo) y **Ventas** (líneas, cliente opcional, detalle de factura).
- `setup-servidor.sh` sincroniza con `origin/main` en lugar de `git pull` (evitaba actualizar por un `package-lock.json` local).
- Migración `002_bodegas_y_consecutivos.sql`: bodega "Principal" por empresa; `orden_produccion.costo_unitario`; índice único en `receta`; `empresa.prefijo_factura`, `ultimo_consecutivo`.
- Pruebas: flujo completo compra → fórmula → producción → traslado → venta verificado en navegador y por API; rechazo de ventas sin existencia.

2026.09.28.1 — Puesta en producción del sistema base

- Repositorios creados en la organización Ecocargapp: `Backend-Agrocarnes` y `Frontend-Agrocarnes`.
- Backend: API Express + PostgreSQL con modelo multiempresa, inventario con costo promedio ponderado (`registrarMovimiento`), compras, traslados, producción por receta, ventas y punto de integración DIAN; autenticación JWT; migraciones con registro (`schema_migrations`).
- Frontend: inicio de sesión y esqueleto de navegación.
- Infraestructura: instalación en el servidor EC2 existente, separada de AgroSoft (base de datos `agrocarnes`, proceso `pm2 agrocarnes-api` en el puerto 4001, nginx propio); dominios `agrocarnes.agrofranpabel.com` y `api-agrocarnes.agrofranpabel.com`; certificado HTTPS Let's Encrypt.
- Script de instalación/actualización idempotente `setup-servidor.sh`.
- Correcciones del mismo día: `ecosystem.config` renombrado a `.cjs` (el proyecto es ESM); `crear_admin.js` actualiza la clave si el usuario existe.
- Migración `001_init.sql`: modelo completo (empresa, usuario, tercero, producto, bodega, existencia, movimiento_inventario, traslado, receta, orden_produccion, compra, compra_item, factura_venta, factura_venta_item) y las tres empresas iniciales.

2026.09.27 — Diseño

- Documento de diseño del sistema (alcance, flujo entre empresas, módulos, modelo de datos, arquitectura, alternativas de facturación electrónica, plan por fases). Decisión: infraestructura propia separada de AgroSoft; facturación electrónica a través de un proveedor tecnológico en lugar de desarrollo propio ante la DIAN.

Pendientes conocidos (no son cambios; se documentan para trazabilidad)

Tema	Estado
Tratamiento tributario de los traslados Agrocarnes → Restaurante / D'Monsa (interno vs. venta intercompañía)	Por confirmar con el contador; el sistema soporta ambos
Campos exactos de NotaCredito/Insert y NotaCredito/Get de Arco	Implementado con la misma convención que Factura/Insert ; por confirmar con Arco y probar contra el simulador antes de la primera nota crédito real
Bitácora de inicios de sesión y consultas	Por implementar
Administración de usuarios desde la aplicación	Por implementar
Copia de respaldos fuera del servidor (S3)	Por implementar
Cuenta de Arco del Restaurante y de D'Monsa	Por configurar
Carga inicial de productos y fórmulas desde las hojas de cálculo existentes	Por hacer